UNIVERSITATEA TEHNICĂ A MOLDOVEI

FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ

DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ

Lucrare de laborator nr. 7

Tehnici de programare

Trusa de unelte HTTP

Realizat de : Bulat Madalina

st. gr. SI-262

Verificat de : Emil Cebotaros

asist. univ.

Chisinau 2026

1. Scopul lucrării

1.1. Scopul lucrării

Scopul lucrării este studierea și aplicarea bibliotecii Python `requests` pentru efectuarea cererilor HTTP, transmiterea și primirea datelor, gestionarea anteturilor, codurilor de stare, redirecționărilor și cookie-urilor. De asemenea, lucrarea urmărește dezvoltarea abilităților de organizare a codului în module separate și de gestionare a erorilor.

2. Realizarea lucrării

2.1. Partea 1. Parametri în URL

Am creat funcțiile `search(params)` și `search_url(params)`, care transmit parametri către server printr-o cerere GET. Prima funcție returnează parametrii primiți de server, iar a doua returnează adresa URL completă, cu parametrii incluși.

2.2. Partea 2. Anteturi HTTP

Am implementat funcțiile `fetch_with_agent(agent)` și `default_agent()`. Acestea permit transmiterea unui antet `User-Agent` personalizat și identificarea User-Agent-ului implicit utilizat de biblioteca `requests`.

2.3. Partea 3. Trimiterea datelor

Am creat funcțiile `post_json(data)` și `post_form(data)`. Prima transmite date către server în format JSON, folosind parametrul `json`, iar a doua transmite date în format formular, folosind parametrul `data`.

2.4. Partea 4. Coduri de stare

Am realizat funcțiile `describe_status(code)` și `is_ok(code)`. Acestea verifică și interpretează codurile de stare HTTP, identificând cererile reușite și erorile clientului sau serverului. Pentru gestionarea erorilor am utilizat `try-except` și metoda `raise_for_status()`.

2.5. Partea 5. Redirecționări

Am implementat funcțiile `count_redirects(n)`, `final_url(n)` și `first_hop(n)`. Acestea determină numărul redirecționărilor, adresa finală după redirecționare și prima adresă indicată de server, fără urmărirea automată a redirecționării.

2.6. Partea 6. Sesiuni și cookie-uri

Am creat funcțiile `cookie_demo(name, value)` și `no_session_demo(name, value)`. Prima utilizează o sesiune `requests.Session()` pentru păstrarea cookie-urilor între cereri, iar a doua utilizează cereri simple, fără păstrarea cookie-urilor.

2.7. Partea 7. Raportul final

Am realizat funcția `report(path)`, care colectează informații despre o pagină web: codul de stare,

adresa URL finală, numărul redirecționărilor, durata cererii și tipul conținutului. Am utilizat un timeout de 3 secunde și am adăugat gestionarea erorilor. În funcția `main()` am testat toate adresele din lista `PATHS`, am afișat paginile reușite în ordinea descrescătoare a timpului de răspuns și am calculat numărul cererilor reușite și eșuate.

2.8. Partea 8. Bonus – Autentificare și module

Am implementat funcția `check_login(user, password)`, care verifică datele de autentificare HTTP Basic și returnează `True` dacă autentificarea este acceptată sau `False` dacă datele sunt greșite. De asemenea, am separat programul în două module: `httptools.py`, care conține funcțiile de lucru, și `main.py`, care importă funcțiile necesare și execută programul.

3. Concluzii

În urma realizării lucrării, am atins scopul propus, studiind și aplicând biblioteca Python `requests` pentru efectuarea cererilor HTTP și comunicarea cu serverele web. Am pus în practică transmiterea parametrilor și a datelor, gestionarea anteturilor, codurilor de stare, redirecționărilor și cookie-urilor.

De asemenea, am dobândit experiență în gestionarea erorilor, verificarea autentificării și organizarea codului în module separate. Prin testarea funcțiilor implementate, am înțeles mai bine modul de funcționare a protocolului HTTP și am consolidat cunoștințele practice de programare în Python.